Architettura del Codice: Fondamenti e Pattern

Una guida a Singleton, Factory, MVC e MVVM nello sviluppo moderno.

Gemini Notebook

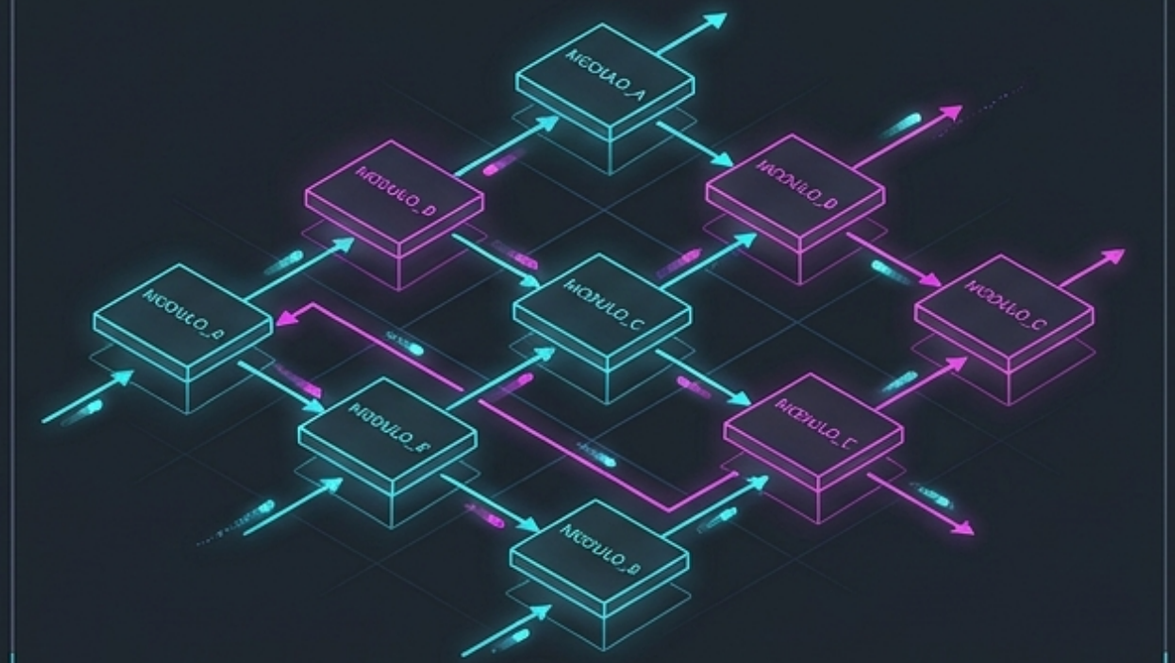
IL PROBLEMA DEL CAOS

Senza un'architettura definita, il codice diventa fragile, strettamente accoppiato e difficile da scalare. I "**Design Pattern**" offrono soluzioni universali e collaudate a problemi ricorrenti. Non sono solo codice, sono **il progetto strutturale** del software.

CODICE SPAGHETTI

FRAGILE, STRETTAMENTE ACCOPPIATO, DIFFICILE DA SCALARE.

MODULI STRUTTURATI



SOLUZIONI UNIVERSALI, COLLAUDATE, PROGETTO STRUTTURALE DEL SOFTWARE.

Gemini Notebook

Le Due Fondamenta del Design

Pattern Creazionali

Gestiscono la logica di creazione degli oggetti, nascondendo la complessità di inizializzazione.

- **Singleton**
- **Factory**

Pattern Architetturali

Definiscono la struttura dell'applicazione e il flusso dei dati tra interfaccia e logica.

- **MVC**
- **MVVM**

Gemini Notebook

Pattern Creazionale: **Singleton**

Garantisce che una classe abbia una sola istanza globale e fornisce un punto di accesso **universale** ad essa.

- Accesso al **database locale** (es. salvataggio note).
- Gestione delle **configurazioni globali dell'app**.
- Sistemi di **logging**.

Gemini Notebook

Il pattern Singleton

Implementazione moderna in .NET:

```
private static readonly
  Lazy<Vault> _instance =
    new(() => new Vault());

public static Vault Instance =>
    _instance.Value;

private Vault() { }
```

Casi d'uso pratici:

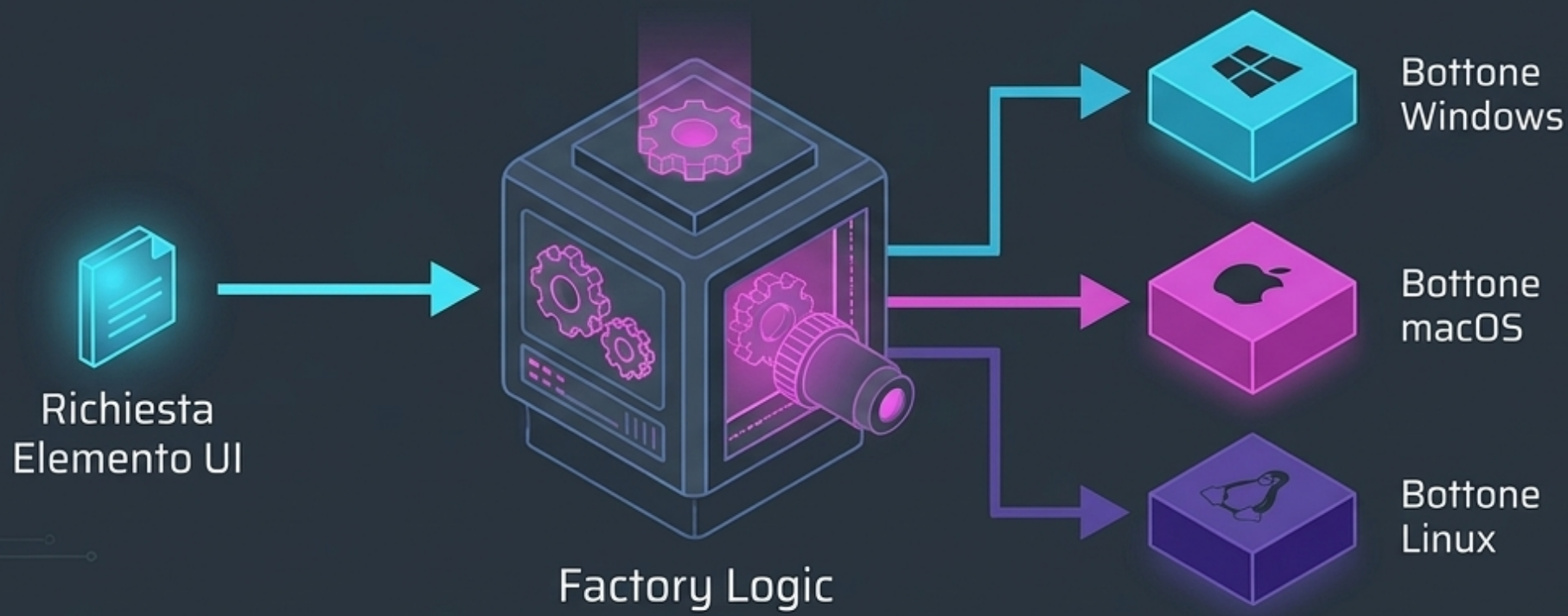
- Gestione della configurazione globale dell'applicazione.
- Sistemi di logging centralizzati.
- Cache applicativa condivisa tra ViewModel.

Gemini Notebook

Pattern Creazionale: Factory Method

Delega la logica di creazione degli oggetti alle sottoclassi o a metodi dedicati, separando il codice che usa l'oggetto da quello che lo crea.

Vantaggio: Estrema flessibilità cross-platform senza modificare la logica principale.



Il controllo assoluto sull'istanziazione

Il Nemico del Disaccoppiamento

```
var service = new CustomerService();
// L'operatore 'new' è colla!
```

L'istanziazione diretta genera codice rigidamente accoppiato.

L'Astrazione Creazionale

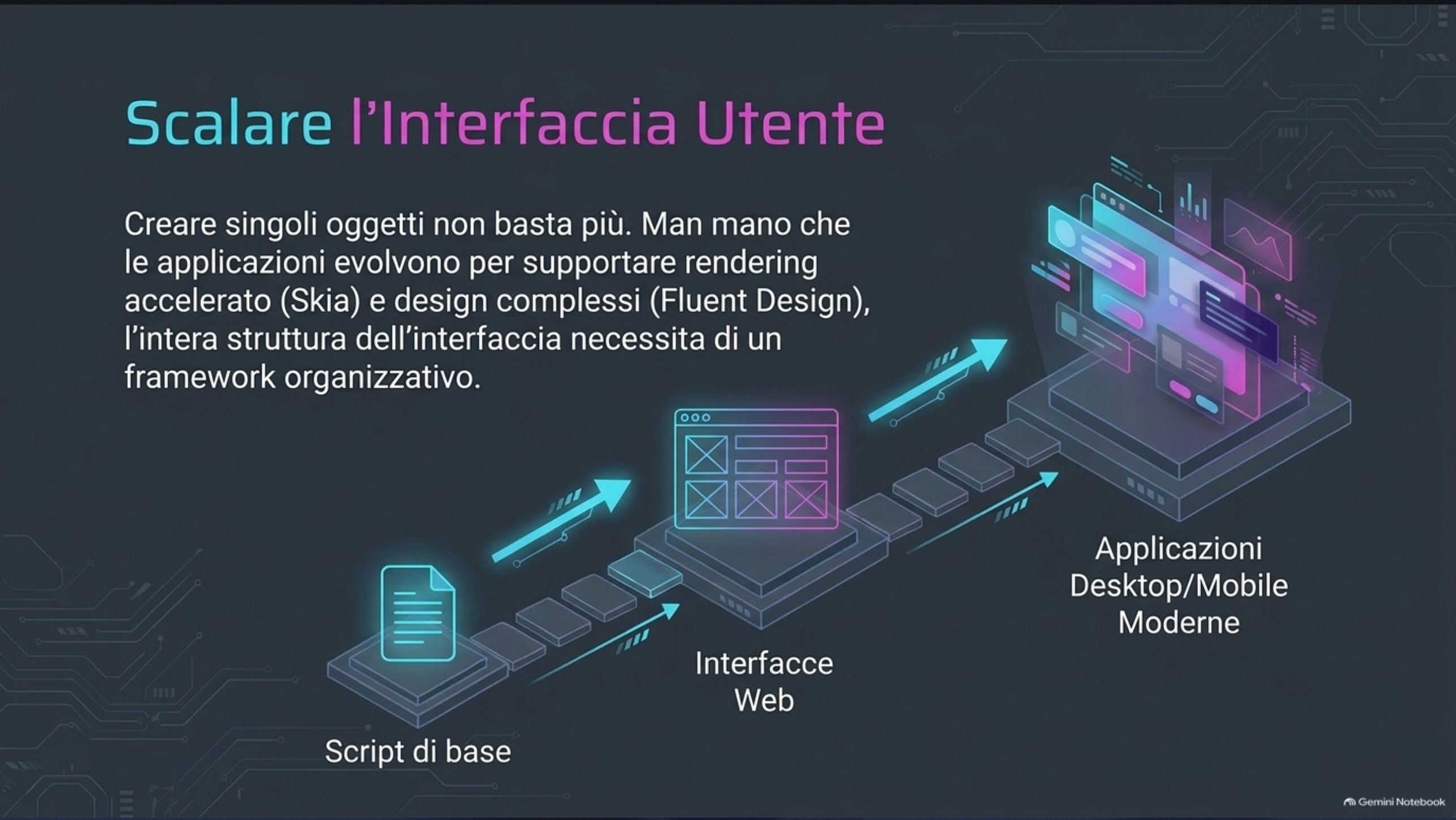
```
var service = Factory.Create();
```

I pattern creazionali astraggono il processo, isolando la logica del client dall'implementazione concreta.

Gemini Notebo

Scalare l'Interfaccia Utente

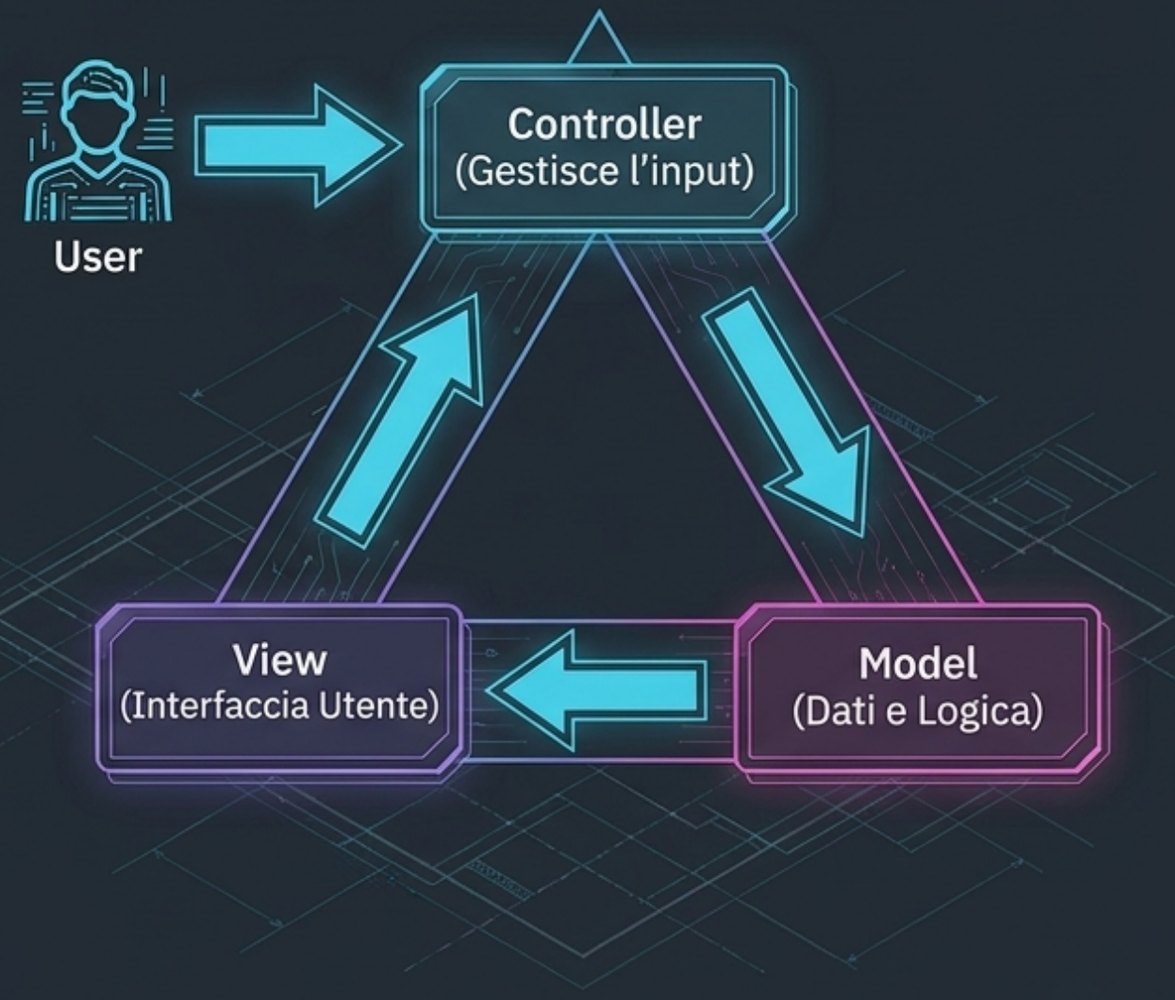
Creare singoli oggetti non basta più. Man mano che le applicazioni evolvono per supportare rendering accelerato (Skia) e design complessi (Fluent Design), l'intera struttura dell'interfaccia necessita di un framework organizzativo.



Pattern Architetturale: MVC (Model-View-Controller)

La separazione classica delle responsabilità. L'utente invia comandi al Controller, che manipola il Model, il quale a sua volta aggiorna la View.

Un pattern solido, ma che richiede un aggiornamento manuale e rigido della UI.



Gemini Notebook

L'Evoluzione: Il Potere del Data Binding

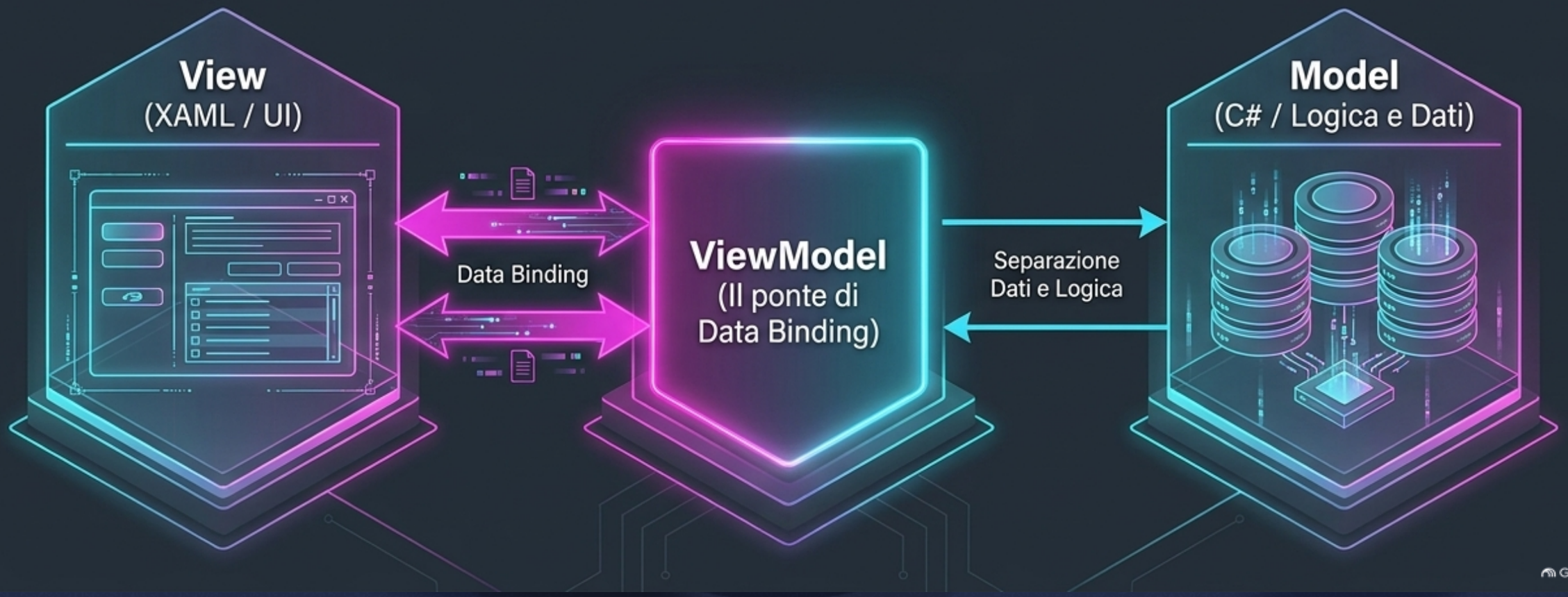
Le interfacce moderne non possono aspettare aggiornamenti manuali. I framework XAML contemporanei introducono il "Data Binding": un ponte automatico che sincronizza istantaneamente i dati sottostanti con ciò che l'utente vede, rendendo il Controller obsoleto.

Gemini Notebook

Pattern Architetturale: MVVM (Model-View-ViewModel)

Lo standard moderno per le interfacce utente (.NET, WPF, WinUI, Avalonia).

Il ViewModel agisce come intermediario invisibile. **Espone i dati del Model** alla View tramite binding, mantenendo un disaccoppiamento totale tra interfaccia grafica e logica di business.



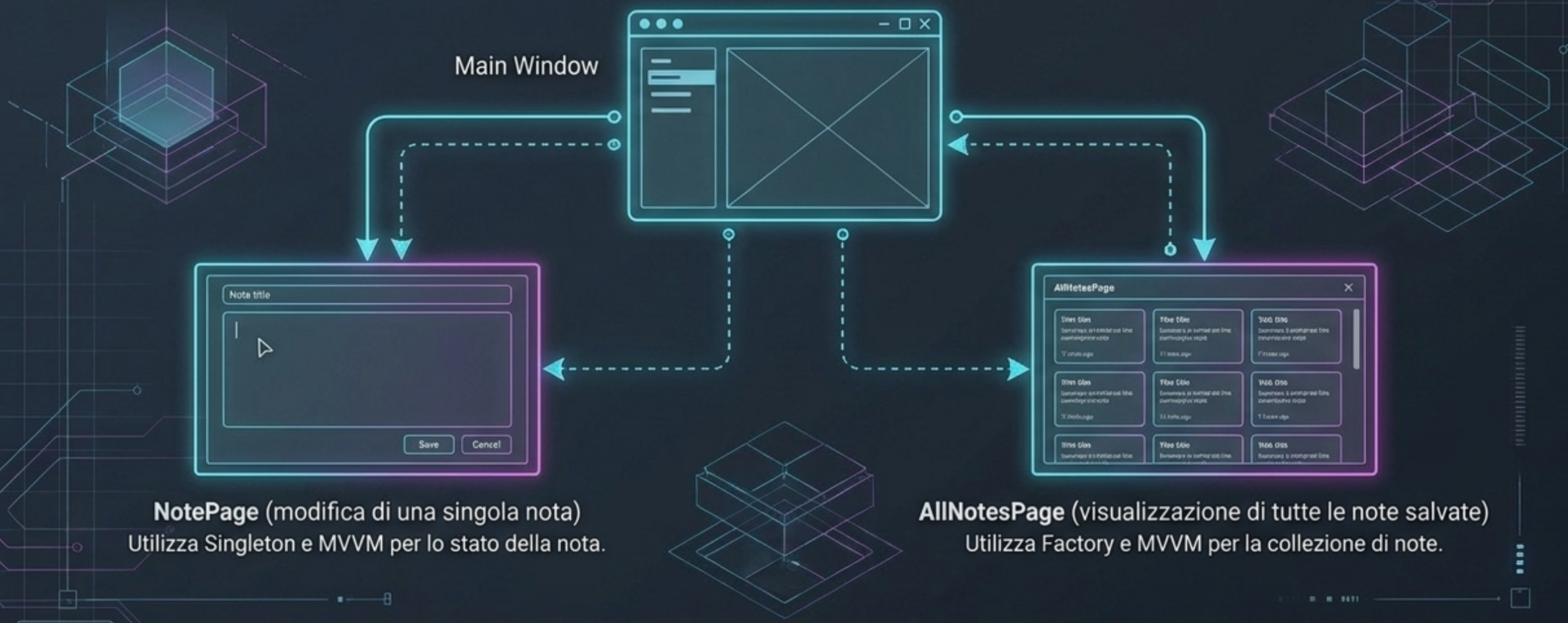
Matrice di diagnosi architetturale: MVC vs MVVM

Vettore Analitico	**MVC** (Model-View-Controller)	**MVVM** (Model-View-ViewModel)
Disaccoppiamento	Basso - Il Controller comanda la View.	Massimo - Ignoranza totale della View.
Gestione dello Stato	Frammentato.	Centralizzato nel ViewModel.
Data Binding in .NET	Non nativo, update manuali.	Fondamentale, engine automatico.
Testabilità (Unit Test)	Difficile - richiede mocking dell'UI.	Assoluta - 100% testabile senza UI.

Gemini Notebook

Caso Studio: La "Notes App"

Costruiamo un'applicazione reale. L'architettura prevede una navigazione fluida tra viste diverse. Ma come si uniscono **Singleton**, **Factory** e **MVVM** in questo singolo progetto?

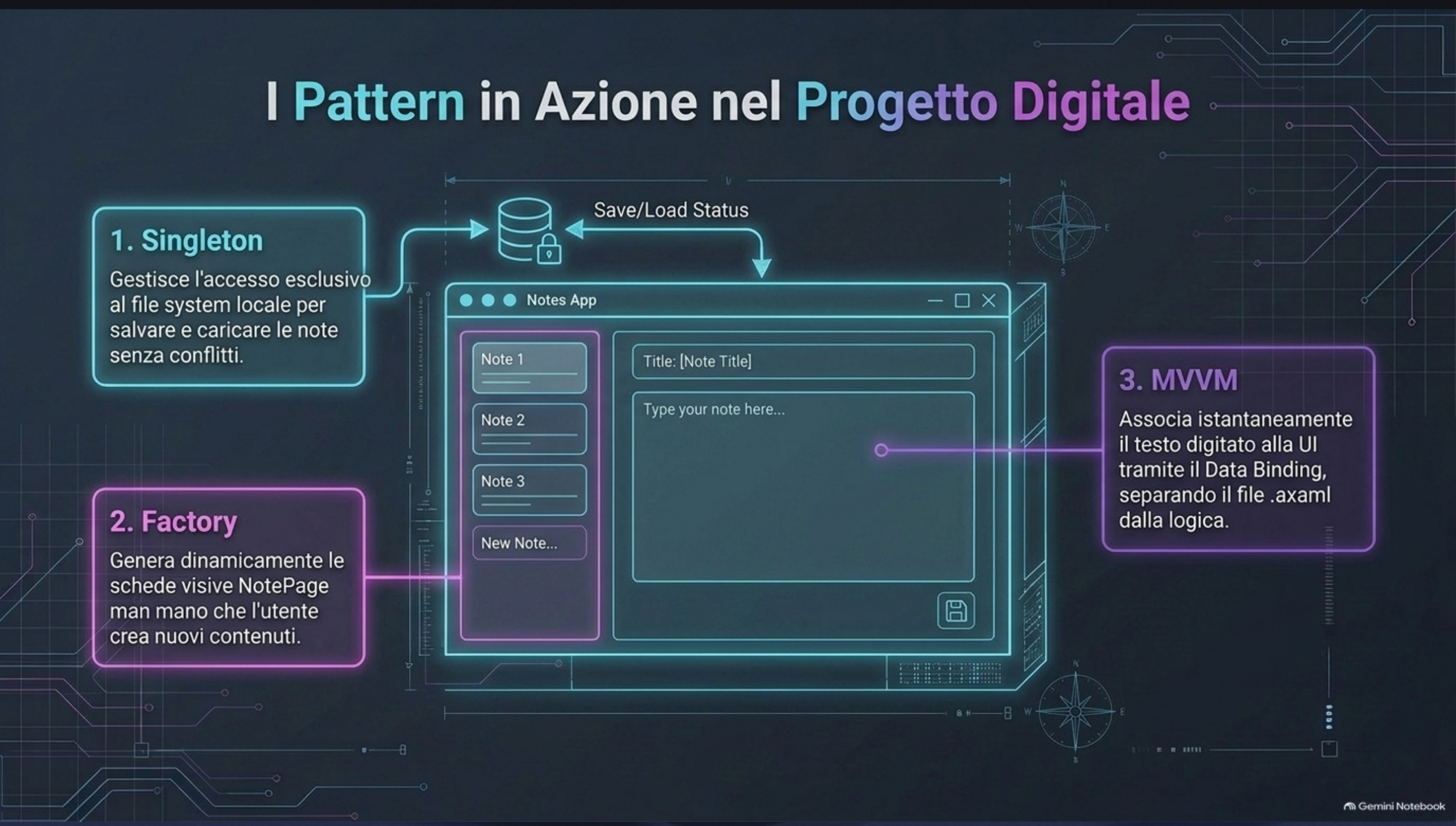


NotePage (modifica di una singola nota)
Utilizza Singleton e MVVM per lo stato della nota.

AllNotesPage (visualizzazione di tutte le note salvate)
Utilizza Factory e MVVM per la collezione di note.

Gemini Notebook

I Pattern in Azione nel Progetto Digitale



L’Infrastruttura del Codice

Il Data Binding unisce i due mondi senza creare dipendenze rigide.

XAML

```
<TextBlock
    Text="{Binding Greeting}"
    HorizontalAlignment="Center"
/>
```

(Il markup dichiarativo che definisce la View)

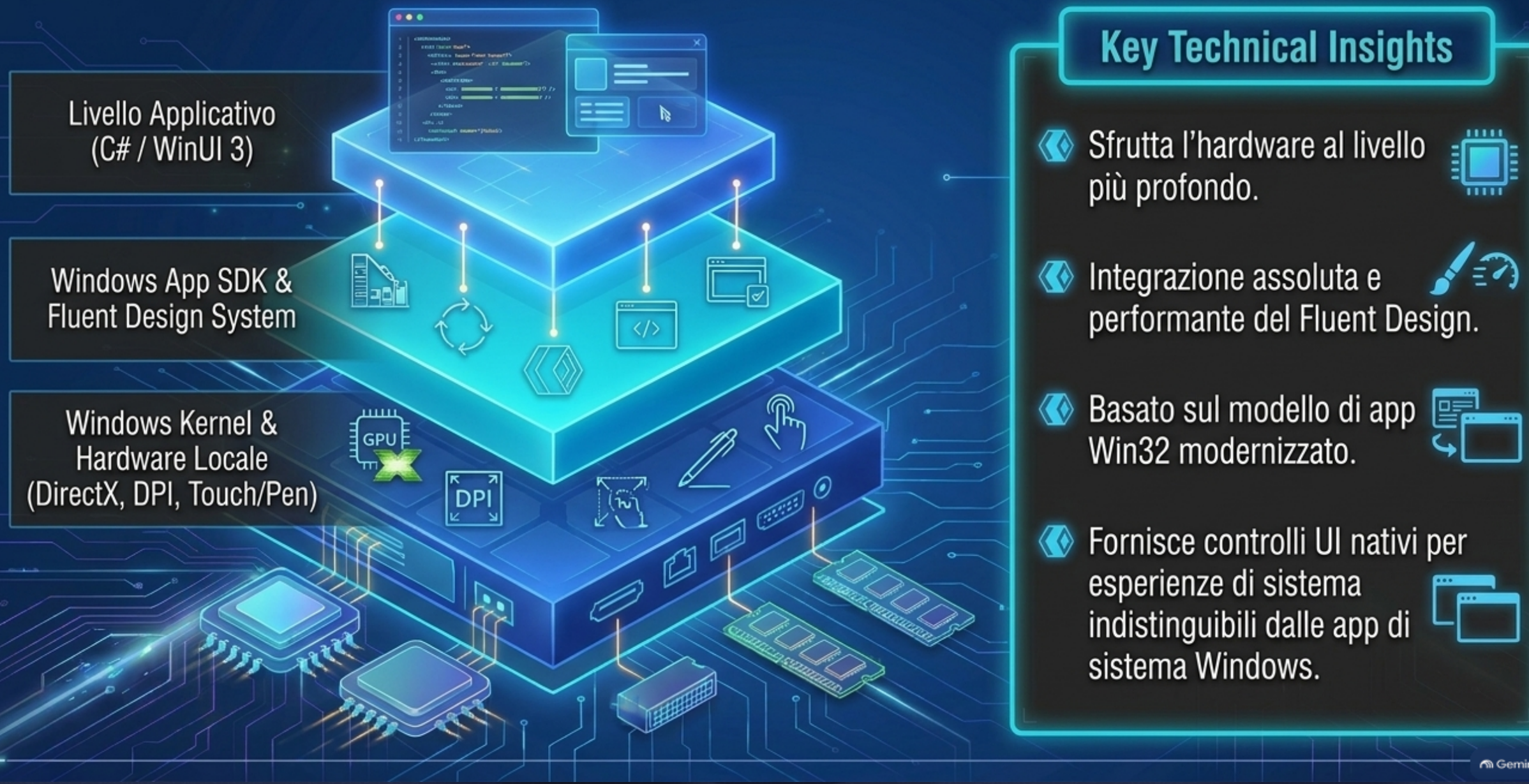
C# ViewModel

```
public string Greeting
    => "Hello, WinUI!";
```

(La logica sottostante disaccoppiata)

Gemini Notebook

L’esperienza Nativa Windows: WinUI 3



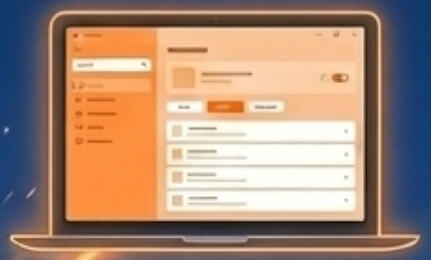
Key Technical Insights

- Sfrutta l’hardware al livello più profondo.
- Integrazione assoluta e performante del Fluent Design.
- Basato sul modello di app Win32 modernizzato.
- Fornisce controlli UI nativi per esperienze di sistema indistinguibili dalle app di sistema Windows.

Gemini Notebook

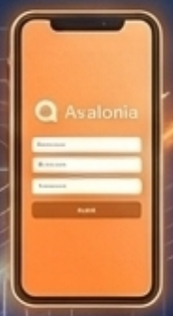
La potenza Skia senza confini: Avalonia UI

Linux Monitor

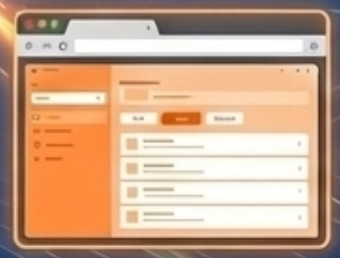


MacBook

C# + XAML Engine



iOS and Android



WebAssembly Browser

Key Technical Insights

- Independent Rendering: Non utilizza i controlli nativi del sistema operativo.
- Disegna l'intera UI in totale autonomia tramite il motore grafico Skia.
- Garantisce consistenza assoluta (Pixel-Perfect) su ogni target, eliminando il jank.
- Impeller è supportato per rendering avanzato su piattaforme come iOS.

Gemini Notebook

La Sinfonia Architetturale .NET (La Sintesi)